Spring Framework — Custom Annotations, AOP & BeanPostProcessor

Custom Annotations • Marker and Configured Annotations • AOP • Bean Lifecycle • Proxy Creation

Study focus: How custom annotations declare method-level behavior, how AOP intercepts those methods, and how Spring's BeanPostProcessor infrastructure participates in proxy creation.

Diagram rule: All flows in these notes are top-to-bottom.

1. Why Do We Need Custom Annotations?

Real applications often need common logic to run before or after methods: logging, authentication, auditing, and execution-time tracking. This is infrastructure or cross-cutting logic. Repeating it inside every business method mixes infrastructure concerns with business logic.

Better approach: AOP + Custom Annotations

```
@TrackExecutionTime
public String getStudent(String data) {
    // Business logic
    return data;
}
```

The annotation declares that a method should be intercepted. The actual interception logic is implemented inside an Aspect.

Key idea: The annotation says what should be intercepted; the Aspect defines what logic should run.

2. What Is a Custom Annotation?

Java allows us to define our own annotations using the special `@interface` syntax.

```
public @interface TrackExecutionTime {
}
```

• An annotation is not a normal class.

• It is declared using @interface.

• It attaches metadata to supported Java program elements.

• An annotation can be a marker or can hold additional information.

3. Marker Annotation

A marker annotation has no properties. It simply marks a method or another supported program element.

```
public @interface TrackExecutionTime {
}

@TrackExecutionTime
public String getStudent() {
    return "Student Data";
}
```

```
@TrackExecutionTime
        ↓
Method is marked
        ↓
AOP checks the annotation
        ↓
Matching Aspect is executed
        ↓
Execution-time logic runs
```

Remember: A marker annotation carries no additional configuration values.

4. Intercepting a Custom Annotation Using AOP

The `@annotation` Pointcut designator matches methods carrying a particular annotation.

```
@Aspect
@Component
public class SimpleAspect {

    @Around("@annotation(com.example.annotation.TrackExecutionTime)")
    public Object measureExecutionTime(
            ProceedingJoinPoint joinPoint) throws Throwable {
```

```
        long startTime = System.currentTimeMillis();

        try {
            return joinPoint.proceed();
        } finally {
            long endTime = System.currentTimeMillis();
            long duration = endTime - startTime;
            System.out.println("Execution Time: " + duration);
        }
    }
}
```

Code	Purpose
@Around	Runs logic before and after target method execution.
@annotation(...)	Matches methods carrying the specified annotation.
ProceedingJoinPoint	Provides access to the intercepted method.
joinPoint.proceed()	Executes the actual target method.
System.currentTimeMillis()	Gets current time in milliseconds.
finally	Executes whether the method succeeds or throws an exception.

The Aspect should not consume the target method's exception. It should allow the exception to propagate to the caller or another exception-handling mechanism.

5. Getting the Method Name

```
String methodName =
        joinPoint.getSignature().getName();

System.out.println(
    "Time Taken By " + methodName + ": " + duration
);
```

Example output:

```
Time Taken By getStudent: 2005
Time Taken By createStudent: 3
```

6. Why Does Around Advice Return the Result?

Different methods can return different types, such as String, Student, Integer, or Long. Using Object allows the Advice to pass through different return types.

```
public Object measureExecutionTime(
        ProceedingJoinPoint joinPoint) throws Throwable {

    return joinPoint.proceed();
}
```

Memory trick: @Around controls method execution. It must handle the original method's return value and exception correctly.

7. Annotation Metadata

Java annotations can contain metadata describing the annotation itself. Two important questions are:

• Where can the annotation be applied?

• How long will the annotation information be available?

The two important meta-annotations are `@Target` and `@Retention`. A meta-annotation is an annotation applied to another annotation.

8. @Target — Where Can an Annotation Be Applied?

```
@Target(ElementType.METHOD)
public @interface TrackExecutionTime {
}
```

This restricts @TrackExecutionTime to method declarations.

ElementType	Where it can be applied
TYPE	Class, interface, enum, annotation type, and record
FIELD	Fields

ElementType	Where it can be applied
METHOD	Methods
PARAMETER	Method or constructor parameters
CONSTRUCTOR	Constructors
LOCAL_VARIABLE	Local variables
ANNOTATION_TYPE	Other annotation types
PACKAGE	Packages
TYPE_PARAMETER	Type parameters
TYPE_USE	Type uses
MODULE	Modules
RECORD_COMPONENT	Record components

Multiple locations can be specified:

```
@Target({
    ElementType.METHOD,
    ElementType.TYPE
})
public @interface TrackExecutionTime {
}
```

Important: If @Target is not specified, the annotation can generally be applied wherever the Java language permits that annotation type.

9. @Retention — How Long Is an Annotation Available?

@Retention defines how long annotation information is retained.

Policy	Meaning
SOURCE	Available in source code; discarded by the compiler and not retained for runtime reflection.
CLASS	Retained in the compiled class file, but not required to be available through runtime reflection. This is the default if @Retention is omitted.
RUNTIME	Retained in the compiled class file and available through runtime reflection.

For this Spring AOP custom annotation, the lecture uses RUNTIME because the annotation needs to be inspected at runtime.

```
@Retention(RetentionPolicy.RUNTIME)

SOURCE
        ↓
Source code only

CLASS
        ↓
Compiled class file

RUNTIME
        ↓
Available through runtime reflection
```

10. @Documented

```
@Documented
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
public @interface TrackExecutionTime {
}
```

• It indicates that an annotation should be included in generated Java documentation.

• It does not control where an annotation can be applied.

• It does not control how long an annotation is retained.

11. Complete Custom Annotation — Marker Version

```
@Documented
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
public @interface TrackExecutionTime {
}
```

• Can be applied only to methods.

• Is available at runtime.

• Appears in generated Java documentation.

• Does not contain custom properties.

This is still a marker annotation.

12. Two Ways to Match a Custom Annotation in AOP

Approach 1: Fully qualified annotation name

```
@Around("@annotation(com.example.annotation.TrackExecutionTime)")
public Object measureExecutionTime(
        ProceedingJoinPoint joinPoint) throws Throwable {

    return joinPoint.proceed();
}
```

This provides the complete annotation path. The lecture calls this the type-matching form.

Approach 2: Annotation binding form

```
@Around("@annotation(trackExecutionTime)")
public Object measureExecutionTime(
        ProceedingJoinPoint joinPoint,
        TrackExecutionTime trackExecutionTime) throws Throwable {

    return joinPoint.proceed();
}
```

The annotation instance is bound to the Advice parameter, making its properties accessible. The binding variable name in the Pointcut and the Advice parameter must match.

Type-matching form	Binding form
Uses fully qualified annotation name.	Uses an annotation variable.
Does not require an annotation parameter.	Receives the annotation as a parameter.
Identifies the annotation type.	Identifies and binds the annotation.

13. Marker Annotation vs Configured Annotation

A configured annotation can carry additional information. In this example, it stores a warning threshold and an operation description.

```
public @interface TrackExecutionTime {

    long warnAfter() default 2000;

    String operation() default "";
}
```

Property	Meaning	Default
warnAfter()	Time threshold before warning	2000 milliseconds
operation()	Description of the operation	Empty string

Annotation properties are declared using methods, not normal instance variables.

14. Using a Configured Annotation

```
@TrackExecutionTime(
    warnAfter = 1500,
    operation = "Get Student Data"
)
public String getStudent(String data) {
    return data;
}

@TrackExecutionTime(
    warnAfter = 2000,
    operation = "Creating New Student"
)
public Student createStudent(Student student) {
    return student;
}
```

Each method can have its own configuration. If a value is not provided, the default value is used:

```
@TrackExecutionTime
public void someMethod() {
}

// warnAfter = 2000
// operation = empty string
```

15. Consuming Annotation Properties Inside an Aspect

```
@Around("@annotation(trackExecutionTime)")
public Object measureExecutionTime(
        ProceedingJoinPoint joinPoint,
        TrackExecutionTime trackExecutionTime) throws Throwable {

    long startTime = System.currentTimeMillis();

    try {
        return joinPoint.proceed();
    } finally {
        long endTime = System.currentTimeMillis();
        long duration = endTime - startTime;

        String operation = trackExecutionTime.operation();

        if (operation.isBlank()) {
            operation = joinPoint.getSignature().getName();
        }

        long threshold = trackExecutionTime.warnAfter();

        if (duration >= threshold) {
            System.out.println("SLOW OPERATION ALERT");
            System.out.println(
                "Time Taken By " + operation + ": " + duration
            );
        } else {
            System.out.println(
                "Time Taken By " + operation + ": " + duration
            );
        }
    }
}
```

Step	Action
1	Record start time.
2	Execute the target method using joinPoint.proceed().
3	Record end time.
4	Calculate duration.
5	Read operation name from the annotation.
6	Use method name if operation is empty.
7	Read warning threshold.
8	Compare duration with threshold and print the appropriate message.

```
Method is called
        ↓
Around Advice starts
        ↓
Read annotation properties
```

```
        ↓
Record start time
        ↓
Execute target method
        ↓
Record end time
        ↓
Calculate duration
        ↓
Compare duration with threshold
        ↓
Print warning or normal timing
```

Example output:

```
SLOW OPERATION ALERT
Time Taken By Get Student Data: 2005

Time Taken By Creating New Student: 5
```

16. Complete Custom Annotation — Configured Version

```
@Documented
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
public @interface TrackExecutionTime {

    long warnAfter() default 2000;

    String operation() default "";
}
```

• @Target restricts where the annotation can be applied.

• @Retention controls how long annotation information is retained.

• @Documented includes it in generated Java documentation.

• warnAfter() provides the warning threshold.

• operation() provides the operation description.

17. BeanPostProcessor

Spring introduces two additional Bean lifecycle stages: BeanPostProcessor before initialization and BeanPostProcessor after initialization.

BeanPostProcessor is a Spring interface that allows a Bean to be processed before and after its initialization callbacks.

```
postProcessBeforeInitialization()
postProcessAfterInitialization()
```

Both methods receive the Bean object and Bean name, and both return a Bean object. Spring's AOP infrastructure uses BeanPostProcessor processing to participate in creating proxies for Beans that need interception.

18. Bean Lifecycle with BeanPostProcessor

The extended sequence discussed in the lecture:

```
Read Bean Definition
        ↓
Instantiate object
        ↓
Inject dependencies
        ↓
Run Aware callbacks
        ↓
BeanPostProcessor before initialization
        ↓
Run initialization callbacks
        ↓
BeanPostProcessor after initialization
        ↓
Store final Bean reference
```

@PostConstruct is part of the initialization callback phase. It runs after the before-initialization BeanPostProcessor phase and before the after-initialization phase.

19. BeanPostProcessor Before Initialization

```
postProcessBeforeInitialization()
```

It executes before initialization callbacks. The lecture describes possible uses such as preparing a Bean, validating it, providing initial configurations, and making it ready for subsequent processing.

```
Bean Created
        ↓
Dependencies Injected
        ↓
Aware Callbacks
        ↓
BeanPostProcessor Before Initialization
        ↓
Initialization Callbacks
```

20. BeanPostProcessor After Initialization

```
postProcessAfterInitialization()
```

It executes after initialization callbacks. This is the important stage discussed in relation to Spring AOP. Spring checks whether the Bean needs to be wrapped in a proxy. If a matching Aspect applies, the AOP infrastructure can return a proxy instead of the original target object.

21. How Does Spring Decide Whether to Create a Proxy?

```
@Service
public class StudentService {

    @TrackExecutionTime
    public String getStudent() {
        return "Student Data";
    }
}
```

When StudentService is processed, Spring checks whether any applicable Aspect matches its methods.

Case 1: An Aspect applies

```
StudentService Bean Created
        ↓
Dependencies Injected
        ↓
Initialization Completed
        ↓
BeanPostProcessor Checks Aspects
        ↓
Does an Aspect Apply?
        ↓
Yes
        ↓
Create AOP Proxy
        ↓
Return Proxy Bean
```

Case 2: No Aspect applies

```
StudentService Bean Created
        ↓
Dependencies Injected
        ↓
Initialization Completed
        ↓
BeanPostProcessor Checks Aspects
        ↓
Does an Aspect Apply?
        ↓
No
        ↓
Return Original Target Object
```

In both cases, Spring returns a Bean reference. The difference is whether that reference is a proxy or the original target object.

22. Proxy vs Target Object

Term	Meaning
Target Object	The original object containing business logic.
Proxy Object	The object that intercepts method calls and applies AOP logic.

```
StudentController
        ↓
Proxy Object
        ↓
AOP Advice
```

```
        ↓
Target Object
        ↓
Actual Business Method
```

The Controller receives the proxy reference when Spring creates one for the service. The proxy intercepts the call and delegates to the target through the AOP invocation chain.

23. What Happens When the Controller Uses StudentService?

```
@RestController
public class StudentController {

    private final StudentService studentService;

    public StudentController(StudentService studentService) {
        this.studentService = studentService;
    }
}
```

If Spring determines that StudentService needs a proxy, the injected reference is the proxy. The Controller does not need to know whether it received a proxy or the original target object.

```
Spring Creates StudentService
        ↓
BeanPostProcessor Processes It
        ↓
Aspect Matching Check
        ↓
Proxy Created
        ↓
Proxy Returned as Bean
        ↓
StudentController Receives Proxy
        ↓
Controller Calls Service Method
        ↓
AOP Advice Executes
        ↓
Target Method Executes
```

24. When Does Spring Identify Pointcuts and Aspects?

The lecture explains that Spring processes Aspect and Pointcut information during application startup and prepares matching information in an efficient data structure. This avoids performing a complete search through every Aspect for every method invocation.

Startup-time processing

```
Spring Application Starts
        ↓
Aspects Are Identified
        ↓
Advice and Pointcut Information Is Processed
        ↓
Matching Information Is Prepared
        ↓
Bean Creation and Proxy Processing
```

When a method executes

```
Method Is Called
        ↓
Check Applicable Interception
        ↓
Matching Pointcut Found?
        ↓
Yes
        ↓
Execute Advice
        ↓
Execute Target Method
```

25. Most Important Annotations and Interfaces

Annotation / Interface	Purpose
@interface	Declares a custom annotation.
@Target	Restricts where an annotation can be applied.
@Retention	Defines how long annotation information is retained.
@Documented	Includes annotation information in generated documentation.
@Around	Runs logic around target method execution.
@annotation	Matches methods carrying a specified annotation.
@TrackExecutionTime	Custom annotation created in the lecture.
BeanPostProcessor	Allows processing before and after Bean initialization.
ProceedingJoinPoint	Provides control over intercepted method execution.

26. Interview Questions

Q1. How do you create a custom annotation in Java?

Using @interface.

Q2. What is a marker annotation?

An annotation without properties that marks a supported program element.

Q3. What is a configured annotation?

An annotation containing properties to hold additional configuration values.

Q4. What is @Target?

It specifies where an annotation can be applied.

Q5. What is @Retention?

It defines how long annotation information is retained.

Q6. What are the three retention policies?

SOURCE, CLASS, and RUNTIME.

Q7. Which retention policy is required for runtime annotation inspection?

RetentionPolicy.RUNTIME.

Q8. What is @Documented?

It indicates that an annotation should appear in generated Java documentation.

Q9. What is the difference between @annotation and annotation binding?

Type matching identifies an annotation by its fully qualified name; binding also makes the annotation instance available as an Advice parameter.

Q10. Why do we use ProceedingJoinPoint?

It provides access to the intercepted method and allows the Advice to proceed with target execution.

Q11. Why does Around Advice commonly return Object?

The intercepted method can return different types; returning proceed() preserves the original return value.

Q12. Why use finally in execution-time tracking?

To measure execution time even when the target method throws an exception.

Q13. What is BeanPostProcessor?

A Spring interface that allows Beans to be processed before and after initialization callbacks.

Q14. What are its two important methods?

postProcessBeforeInitialization() and postProcessAfterInitialization().

Q15. How is BeanPostProcessor related to Spring AOP?

Spring's AOP infrastructure uses BeanPostProcessor processing to create proxies for Beans that need interception.

Q16. What is a target object?

The original object containing the actual business logic.

Q17. What is a proxy object?

An object that intercepts method calls and applies additional logic before delegating to the target.

Q18. When does Spring prepare Pointcut matching information?

During application startup, according to the lecture, so matching information can be reused efficiently.

27. Final Revision Flows

Custom Annotation

```
Create Annotation
        ↓
Use @interface
        ↓
Add @Target
        ↓
Add @Retention
        ↓
Add @Documented
        ↓
Add Annotation Properties
        ↓
Apply Annotation to Method
        ↓
Intercept Using AOP
```

AOP Execution-Time Tracking

```
Method Called
        ↓
@Around Advice
        ↓
Record Start Time
        ↓
joinPoint.proceed()
        ↓
Target Method Executes
        ↓
finally Block
        ↓
Record End Time
        ↓
Calculate Duration
        ↓
Compare With Threshold
        ↓
Print Timing or Slow Operation Alert
```

Spring Bean Lifecycle with AOP

```
Read Bean Definition
        ↓
Instantiate Bean
        ↓
Inject Dependencies
        ↓
Aware Callbacks
        ↓
BeanPostProcessor Before Initialization
        ↓
Initialization Callbacks
        ↓
BeanPostProcessor After Initialization
        ↓
Check Applicable Aspects
        ↓
Proxy Required?
        ↓
Return Proxy or Original Target Bean
        ↓
Inject Final Bean Reference
```

28. Things to Remember Before an Interview

• Use @interface to create custom annotations.

• Marker annotations do not contain properties.

• Configured annotations can contain properties declared as methods.

• @Target controls where an annotation can be applied.

• @Retention controls how long annotation information is retained.

• RUNTIME retention allows runtime reflection.

• @annotation can intercept methods carrying a custom annotation.

• Annotation binding lets an Advice access annotation properties.

• @Around allows logic before and after target method execution.

• joinPoint.proceed() executes the target method.

• Return the result of proceed() to preserve the original return value.

• Use finally when timing must be measured even if an exception occurs.

• BeanPostProcessor has before-initialization and after-initialization processing.

• Spring AOP can return a proxy instead of the original target object.

• The Controller receives the final Bean reference from Spring.

Final takeaway: Custom annotations let us declare what should happen on a method, while AOP defines how that behavior is executed. Spring's BeanPostProcessor infrastructure helps create the required proxies during Bean initialization.